Source Code Listing — HSR-AIML-056 Image Quality & Compliance Inspector

src/make_dataset.py

```
"""
Step 1: Build the labelled dataset.

Takes clean e-commerce product images (Fashion Product Images - Small, HF mirror)
and synthetically injects 5 marketplace-style defects with known labels:

    0 clean          - as is
    1 blur           - Gaussian blur (out of focus)
    2 noise          - additive Gaussian + salt&pepper noise (low-light sensor)
    3 low_contrast   - histogram compression (washed-out)
    4 bg_clutter     - non-uniform / textured background (fails white-bg rule)
    5 off_center     - product shifted & shrunk (fails framing rule)

Output: data/images/<class>/<id>.jpg  +  data/labels.csv
"""
import io, os, sys, csv, random
import numpy as np, cv2
import pyarrow.parquet as pq
import urllib.request

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUT = os.path.join(ROOT, "data")
CLASSES = ["clean", "blur", "noise", "low_contrast", "bg_clutter", "off_center"]
N_BASE = 300          # base clean images
SIZE = 256            # working resolution
random.seed(0); np.random.seed(0)

def load_base_images(n):
    cache = os.path.join(ROOT, "data_raw", "shard0.parquet")
    os.makedirs(os.path.dirname(cache), exist_ok=True)
    if not os.path.exists(cache):
        u = "https://huggingface.co/api/datasets/ashraq/fashion-product-images-small/parquet/d
        print("downloading", u)
        urllib.request.urlretrieve(u, cache)
    t = pq.read_table(cache, columns=["id", "image", "masterCategory", "articleType"]).to_pand
    t = t.sample(frac=1, random_state=0).reset_index(drop=True)
    imgs = []
    for _, r in t.iterrows():
        b = r["image"]["bytes"]
        im = cv2.imdecode(np.frombuffer(b, np.uint8), cv2.IMREAD_COLOR)
        if im is None: continue
        # keep only images whose border is near-white (true studio shots)
        border = np.concatenate([im[0], im[-1], im[:, 0], im[:, -1]])
        if border.mean() < 235: continue
        im = pad_to_square(im)
        im = cv2.resize(im, (SIZE, SIZE), interpolation=cv2.INTER_AREA)
        imgs.append((int(r["id"]), im))
        if len(imgs) >= n: break
    return imgs

def pad_to_square(im):
    h, w = im.shape[:2]; s = max(h, w)
    out = np.full((s, s, 3), 255, np.uint8)
    y, x = (s - h)//2, (s - w)//2
```

```
    out[y:y+h, x:x+w] = im
    return out

# ---------------- defect generators ----------------
def blur(im):
    k = random.choice([7, 9, 11, 13])
    return cv2.GaussianBlur(im, (k, k), 0)

def noise(im):
    sigma = random.uniform(15, 35)
    out = im.astype(np.float32) + np.random.normal(0, sigma, im.shape)
    out = np.clip(out, 0, 255).astype(np.uint8)
    p = random.uniform(0.005, 0.02)                        # salt & pepper
    m = np.random.rand(*im.shape[:2])
    out[m < p/2] = 0; out[m > 1 - p/2] = 255
    return out

def low_contrast(im):
    a = random.uniform(0.25, 0.45)                          # contrast gain
    b = random.uniform(100, 140)                            # grey offset
    return np.clip(a * im.astype(np.float32) + b, 0, 255).astype(np.uint8)

def bg_clutter(im):
    """Replace white background with a textured / gradient background."""
    g = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    _, fg = cv2.threshold(g, 240, 255, cv2.THRESH_BINARY_INV)
    fg = cv2.morphologyEx(fg, cv2.MORPH_CLOSE, np.ones((7, 7), np.uint8))
    kind = random.choice(["noise_tex", "gradient", "stripes"])
    h, w = g.shape
    if kind == "noise_tex":
        bg = cv2.GaussianBlur(np.random.randint(60, 200, (h, w, 3)).astype(np.uint8), (0, 0),
    elif kind == "gradient":
        c1 = np.random.randint(40, 220, 3); c2 = np.random.randint(40, 220, 3)
        t = np.linspace(0, 1, w)[None, :, None]
        bg = (c1 * (1 - t) + c2 * t).astype(np.uint8).repeat(h, 0)
    else:
        bg = np.zeros((h, w, 3), np.uint8)
        c1 = np.random.randint(60, 220, 3); c2 = np.random.randint(60, 220, 3)
        for x in range(0, w, 32): bg[:, x:x+16] = c1; bg[:, x+16:x+32] = c2
    mask = (fg > 0)[..., None]
    return np.where(mask, im, bg).astype(np.uint8)

def off_center(im):
    s = random.uniform(0.35, 0.55)
    small = cv2.resize(im, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    h, w = small.shape[:2]
    out = np.full_like(im, 255)
    corner = random.choice([(0, 0), (0, SIZE - w), (SIZE - h, 0), (SIZE - h, SIZE - w)])
    out[corner[0]:corner[0]+h, corner[1]:corner[1]+w] = small
    return out

GEN = {"clean": lambda x: x, "blur": blur, "noise": noise,
       "low_contrast": low_contrast, "bg_clutter": bg_clutter, "off_center": off_center}

if __name__ == "__main__":
    base = load_base_images(N_BASE)
    print("base images:", len(base))
```

```
    rows = []
    for c in CLASSES: os.makedirs(os.path.join(OUT, "images", c), exist_ok=True)
    for pid, im in base:
        for ci, c in enumerate(CLASSES):
            out = GEN[c](im.copy())
            fn = os.path.join("images", c, f"{pid}.jpg")
            cv2.imwrite(os.path.join(OUT, fn), out, [cv2.IMWRITE_JPEG_QUALITY, 92])
            rows.append((fn, c, ci, pid))
    with open(os.path.join(OUT, "labels.csv"), "w", newline="") as f:
        w = csv.writer(f); w.writerow(["path", "label", "label_id", "product_id"]); w.writerow
    print("wrote", len(rows), "images ->", OUT)
```

src/dip_features.py

```
"""
Step 2: Classical DIP feature extraction.

Every function takes a BGR image and returns a scalar quality measure.
`extract_all()` returns the 10-D feature vector used by the rule-based
baseline and the Random-Forest classifier. `intermediates()` returns the
intermediate images (grey, Laplacian, edge map, background mask, bbox) for
the report figures.
"""
import numpy as np, cv2

def to_grey(im):
    return cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)

# ---- 1. Sharpness: variance of Laplacian (second-derivative energy) ----
def sharpness(g):
    return float(cv2.Laplacian(g, cv2.CV_64F).var())

# ---- 2. Noise: MAD of high-pass residual (image - median filtered) ----
def noise_level(g):
    res = g.astype(np.float32) - cv2.medianBlur(g, 3).astype(np.float32)
    return float(1.4826 * np.median(np.abs(res - np.median(res))))

# ---- 3. Salt & pepper fraction: pixels at 0 or 255 that differ from median ----
def sp_fraction(g):
    med = cv2.medianBlur(g, 3)
    hit = ((g == 0) | (g == 255)) & (np.abs(g.astype(int) - med.astype(int)) > 100)
    return float(hit.mean())

# ---- 4/5. Contrast: std-dev of grey + 1-99 percentile histogram spread ----
def contrast_std(g):
    return float(g.std())

def hist_spread(g):
    h = cv2.calcHist([g], [0], None, [256], [0, 256]).ravel()
    cdf = np.cumsum(h) / h.sum()
    lo = np.searchsorted(cdf, 0.01); hi = np.searchsorted(cdf, 0.99)
    return float(hi - lo)

# ---- 6. Foreground mask via Otsu + morphology; border-white fraction ----
def fg_mask(g):
    # product is darker than white studio bg -> invert threshold
    _, m = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    k = np.ones((5, 5), np.uint8)
```

```
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, k)
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, k)
    # keep largest connected component
    n, lab, stats, _ = cv2.connectedComponentsWithStats(m)
    if n > 1:
        big = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
        m = np.where(lab == big, 255, 0).astype(np.uint8)
    return m

def border_white_frac(g, t=235):
    b = np.concatenate([g[:8].ravel(), g[-8:].ravel(), g[:, :8].ravel(), g[:, -8:].ravel()])
    return float((b > t).mean())

# ---- 7. Background uniformity: std of grey outside the foreground mask ----
def bg_std(g, m):
    bg = g[m == 0]
    return float(bg.std()) if bg.size > 100 else 0.0

# ---- 8/9/10. Region features from bounding box of the foreground ----
def region_feats(m):
    h, w = m.shape
    ys, xs = np.where(m > 0)
    if len(xs) == 0: return 0.0, 0.5, 0.0
    x0, x1, y0, y1 = xs.min(), xs.max(), ys.min(), ys.max()
    area_frac = (x1 - x0 + 1) * (y1 - y0 + 1) / (h * w)
    cx, cy = (x0 + x1) / 2 / w, (y0 + y1) / 2 / h
    centroid_off = float(np.hypot(cx - 0.5, cy - 0.5))          # 0 = centred
    fill = float((m > 0).mean())
    return float(area_frac), centroid_off, fill

FEATURE_NAMES = ["sharpness", "noise_mad", "sp_frac", "contrast_std", "hist_spread",
                 "border_white", "bg_std", "bbox_area", "centroid_off", "fg_fill"]

def extract_all(im):
    g = to_grey(im)
    m = fg_mask(g)
    a, c, f = region_feats(m)
    return np.array([sharpness(g), noise_level(g), sp_fraction(g), contrast_std(g),
                     hist_spread(g), border_white_frac(g), bg_std(g, m), a, c, f], np.float32)

def intermediates(im):
    g = to_grey(im)
    lap = cv2.convertScaleAbs(cv2.Laplacian(g, cv2.CV_64F))
    edges = cv2.Canny(g, 50, 150)
    m = fg_mask(g)
    box = im.copy()
    ys, xs = np.where(m > 0)
    if len(xs):
        cv2.rectangle(box, (xs.min(), ys.min()), (xs.max(), ys.max()), (0, 0, 255), 2)
    hist = cv2.calcHist([g], [0], None, [256], [0, 256]).ravel()
    return {"grey": g, "laplacian": lap, "edges": edges, "mask": m, "bbox": box, "hist": hist}
```

src/train_eval.py

```
"""
Step 3: Baseline (hand-tuned rules on DIP features)  vs
        Improved-1 (Random Forest on the same 10 DIP features)  vs
        Improved-2 (small CNN on raw 96x96 pixels)
```

```
Split is BY PRODUCT (all 6 variants of a product go to the same split) so
the models never see a test product during training.
Outputs: results/metrics.json, results/*.png, results/features.csv
"""
import os, json, time, csv
import numpy as np, cv2, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import (f1_score, accuracy_score, confusion_matrix,
                             classification_report)
import torch, torch.nn as nn
from dip_features import extract_all, FEATURE_NAMES, intermediates

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "data"); RES = os.path.join(ROOT, "results")
os.makedirs(RES, exist_ok=True)
CLASSES = ["clean", "blur", "noise", "low_contrast", "bg_clutter", "off_center"]
torch.manual_seed(0); np.random.seed(0)

# ---------------------------------------------------------------- data
df = pd.read_csv(os.path.join(DATA, "labels.csv"))
imgs = [cv2.imread(os.path.join(DATA, p)) for p in df.path]
t0 = time.time()
X = np.stack([extract_all(im) for im in imgs])
feat_ms = (time.time() - t0) / len(imgs) * 1000
y = df.label_id.values; groups = df.product_id.values
pd.DataFrame(X, columns=FEATURE_NAMES).assign(label=df.label).to_csv(
    os.path.join(RES, "features.csv"), index=False)

tr, te = next(GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=0).split(X, y, groups))
print(f"train {len(tr)}  test {len(te)}  feature time {feat_ms:.2f} ms/img")

# ---------------------------------------------------------------- baseline: rules
# thresholds set by eye from the TRAIN-split feature distributions (see per-class means)
F = {n: i for i, n in enumerate(FEATURE_NAMES)}
def rule_predict(x):
    if x[F["noise_mad"]] > 3.0:                                  return 2   # noise
    if x[F["hist_spread"]] < 100:                                return 3   # low contrast
    if x[F["border_white"]] < 0.3:                               return 4   # cluttered bg
    if x[F["bbox_area"]] < 0.12 and x[F["centroid_off"]] > 0.25: return 5 # off-centre
    if x[F["sharpness"]] < 30:                                   return 1   # blur
    return 0                                                             # clean

# print per-class feature means on train split (for the paper table)
means = pd.DataFrame(X[tr], columns=FEATURE_NAMES).groupby(df.label.values[tr]).mean().loc[CLA
means.round(3).to_csv(os.path.join(RES, "feature_means_by_class.csv"))
print(means.round(2))

yp_rule = np.array([rule_predict(x) for x in X[te]])

# ---------------------------------------------------------------- improved 1: RF on DIP featu
rf = RandomForestClassifier(n_estimators=300, random_state=0, n_jobs=-1)
t0 = time.time(); rf.fit(X[tr], y[tr]); rf_train_s = time.time() - t0
yp_rf = rf.predict(X[te])
```

```
imp = pd.Series(rf.feature_importances_, index=FEATURE_NAMES).sort_values(ascending=False)
imp.to_csv(os.path.join(RES, "rf_feature_importance.csv"))

# ---------------------------------------------------------------- improved 2: small CNN on pi
def to_tensor(ims):
    a = np.stack([cv2.resize(im, (96, 96), interpolation=cv2.INTER_AREA) for im in ims])
    a = a[..., ::-1].astype(np.float32) / 255.0                # BGR->RGB
    return torch.tensor(a.transpose(0, 3, 1, 2).copy())
Xt_tr, Xt_te = to_tensor([imgs[i] for i in tr]), to_tensor([imgs[i] for i in te])
yt_tr = torch.tensor(y[tr])

class SmallCNN(nn.Module):
    def __init__(s, n=6):
        super().__init__()
        def blk(i, o): return nn.Sequential(nn.Conv2d(i, o, 3, padding=1), nn.BatchNorm2d(o),
        s.f = nn.Sequential(blk(3, 16), blk(16, 32), blk(32, 64), blk(64, 64))    # 96->6x6
        s.h = nn.Sequential(nn.Flatten(), nn.Dropout(0.3), nn.Linear(64 * 6 * 6, 128), nn.ReLU
    def forward(s, x): return s.h(s.f(x))

cnn = SmallCNN(); opt = torch.optim.Adam(cnn.parameters(), 1e-3); lossf = nn.CrossEntropyLoss(
EPOCHS, BS = 20, 64
t0 = time.time(); hist = []
for ep in range(EPOCHS):
    cnn.train(); perm = torch.randperm(len(Xt_tr)); tot = 0
    for i in range(0, len(perm), BS):
        idx = perm[i:i+BS]
        xb = Xt_tr[idx]
        if torch.rand(1).item() < 0.5: xb = xb.flip(3)                        # horizontal-flip
        opt.zero_grad(); l = lossf(cnn(xb), yt_tr[idx]); l.backward(); opt.step()
        tot += l.item() * len(idx)
    hist.append(tot / len(perm)); print(f"epoch {ep+1} loss {hist[-1]:.3f}")
cnn_train_s = time.time() - t0
cnn.eval()
with torch.no_grad():
    yp_cnn = cnn(Xt_te).argmax(1).numpy()
    cnn_train_acc = float((cnn(Xt_tr).argmax(1).numpy() == y[tr]).mean())
print("CNN train acc", cnn_train_acc)

# ---------------------------------------------------------------- metrics
def summarise(name, yp):
    return {"model": name,
            "accuracy": round(float(accuracy_score(y[te], yp)), 4),
            "macro_f1": round(float(f1_score(y[te], yp, average="macro")), 4),
            "per_class_f1": dict(zip(CLASSES, np.round(f1_score(y[te], yp, average=None), 3).t
            # binary PASS/FAIL view: clean vs any defect
            "defect_recall": round(float(((yp != 0) & (y[te] != 0)).sum() / (y[te] != 0).sum()
            "false_reject_rate": round(float(((yp != 0) & (y[te] == 0)).sum() / (y[te] == 0).s
results = [summarise("Rule baseline (DIP features)", yp_rule),
           summarise("Random Forest (DIP features)", yp_rf),
           summarise("Small CNN (raw pixels)", yp_cnn)]
meta = {"n_train": int(len(tr)), "n_test": int(len(te)), "feature_ms_per_img": round(feat_ms,
        "rf_train_s": round(rf_train_s, 2), "cnn_train_s": round(cnn_train_s, 1), "cnn_epochs"
        "rf_importance": imp.round(4).to_dict()}
json.dump({"results": results, "meta": meta}, open(os.path.join(RES, "metrics.json"), "w"), in
for r in results: print(r["model"], "acc", r["accuracy"], "macroF1", r["macro_f1"])
print(classification_report(y[te], yp_rf, target_names=CLASSES))
```

```
# ---------------------------------------------------------------- figures
def plot_cm(yp, title, fn):
    cm = confusion_matrix(y[te], yp)
    fig, ax = plt.subplots(figsize=(4.2, 3.8))
    ax.imshow(cm, cmap="Blues"); ax.set_title(title, fontsize=10)
    ax.set_xticks(range(6)); ax.set_yticks(range(6))
    ax.set_xticklabels(CLASSES, rotation=45, ha="right", fontsize=8); ax.set_yticklabels(CLASS
    for i in range(6):
        for j in range(6):
            ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=8,
                    color="white" if cm[i, j] > cm.max()/2 else "black")
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); fig.tight_layout(); fig.savefig(os.path
plot_cm(yp_rule, "Rule baseline", "cm_rule.png")
plot_cm(yp_rf, "Random Forest", "cm_rf.png")
plot_cm(yp_cnn, "Small CNN", "cm_cnn.png")

# feature importance bar
fig, ax = plt.subplots(figsize=(4.2, 2.6)); imp[::-1].plot.barh(ax=ax, color="#3b6ea5")
ax.set_xlabel("RF importance"); fig.tight_layout(); fig.savefig(os.path.join(RES, "rf_importan

# intermediate DIP stages for one product (all 6 variants)
pid = df.product_id.iloc[te[0]]
fig, axes = plt.subplots(6, 6, figsize=(9, 9))
cols = ["input", "grey", "Laplacian", "Canny edges", "FG mask (Otsu+morph)", "bbox"]
for r, c in enumerate(CLASSES):
    im = cv2.imread(os.path.join(DATA, "images", c, f"{pid}.jpg")); it = intermediates(im)
    panels = [im[..., ::-1], it["grey"], it["laplacian"], it["edges"], it["mask"], it["bbox"][
    for k, p in enumerate(panels):
        axes[r, k].imshow(p, cmap=None if p.ndim == 3 else "gray", vmin=None if p.ndim == 3 el
        if r == 0: axes[r, k].set_title(cols[k], fontsize=8)
    axes[r, 0].text(-0.1, 0.5, c, transform=axes[r, 0].transAxes, rotation=90, va="center", fo
fig.tight_layout(); fig.savefig(os.path.join(RES, "intermediate_stages.png"), dpi=170); plt.cl

# histograms: clean vs low-contrast vs noise
fig, ax = plt.subplots(figsize=(4.2, 2.4))
for c, col in [("clean", "k"), ("low_contrast", "tab:orange"), ("noise", "tab:green")]:
    it = intermediates(cv2.imread(os.path.join(DATA, "images", c, f"{pid}.jpg")))
    ax.plot(it["hist"] / it["hist"].sum(), col, label=c, lw=1)
ax.set_yscale("log"); ax.set_xlabel("grey level"); ax.set_ylabel("p (log)"); ax.legend(fontsiz
fig.savefig(os.path.join(RES, "histograms.png"), dpi=200); plt.close(fig)

# feature distributions per class (box plots for 4 key features)
fig, axes = plt.subplots(1, 4, figsize=(9, 2.3))
for ax, f in zip(axes, ["sharpness", "noise_mad", "hist_spread", "centroid_off"]):
    ax.boxplot([X[y == i, F[f]] for i in range(6)], showfliers=False)
    ax.set_xticks(range(1, 7)); ax.set_xticklabels(CLASSES, rotation=60, fontsize=6); ax.set_t
    if f == "sharpness": ax.set_yscale("log")
fig.tight_layout(); fig.savefig(os.path.join(RES, "feature_boxplots.png"), dpi=200); plt.close

# failure cases: RF mistakes
wrong = np.where(yp_rf != y[te])[0][:6]
if len(wrong):
    fig, axes = plt.subplots(1, len(wrong), figsize=(1.6 * len(wrong), 2.0))
    axes = np.atleast_1d(axes)
    for ax, w in zip(axes, wrong):
        ax.imshow(imgs[te[w]][..., ::-1]); ax.axis("off")
        ax.set_title(f"T:{CLASSES[y[te][w]]}\nP:{CLASSES[yp_rf[w]]}", fontsize=7)
```

```
    fig.tight_layout(); fig.savefig(os.path.join(RES, "failure_cases_rf.png"), dpi=200); plt.c
wrong = np.where(yp_rule != y[te])[0]
sel = wrong[np.linspace(0, len(wrong) - 1, min(6, len(wrong))).astype(int)] if len(wrong) else
if len(sel):
    fig, axes = plt.subplots(1, len(sel), figsize=(1.6 * len(sel), 2.0)); axes = np.atleast_1d
    for ax, w in zip(axes, sel):
        ax.imshow(imgs[te[w]][..., ::-1]); ax.axis("off")
        ax.set_title(f"T:{CLASSES[y[te][w]]}\nP:{CLASSES[yp_rule[w]]}", fontsize=7)
    fig.tight_layout(); fig.savefig(os.path.join(RES, "failure_cases_rule.png"), dpi=200); plt

# sample dataset strip
fig, axes = plt.subplots(2, 6, figsize=(9, 3.2))
for r, p in enumerate(df.product_id.unique()[:2]):
    for k, c in enumerate(CLASSES):
        axes[r, k].imshow(cv2.imread(os.path.join(DATA, "images", c, f"{p}.jpg"))[..., ::-1]);
        if r == 0: axes[r, k].set_title(c, fontsize=9)
fig.tight_layout(); fig.savefig(os.path.join(RES, "dataset_samples.png"), dpi=170); plt.close(
torch.save(cnn.state_dict(), os.path.join(RES, "small_cnn.pt"))
print("done")
```

src/ablation_and_diagram.py

```
"""Step 4: ablations (feature groups, morphology on/off) + pipeline block diagram."""
import os, json, numpy as np, pandas as pd, cv2
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import f1_score
import dip_features as D

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RES = os.path.join(ROOT, "results"); DATA = os.path.join(ROOT, "data")
df = pd.read_csv(os.path.join(DATA, "labels.csv"))
X = pd.read_csv(os.path.join(RES, "features.csv"))[D.FEATURE_NAMES].values
y = df.label_id.values; g = df.product_id.values
tr, te = next(GroupShuffleSplit(1, test_size=0.3, random_state=0).split(X, y, g))
F = {n: i for i, n in enumerate(D.FEATURE_NAMES)}

groups = {"sharpness only": ["sharpness"],
          "global stats (sharp+noise+contrast)": ["sharpness", "noise_mad", "sp_frac", "contra
          "+ background (border/bg_std)": ["sharpness", "noise_mad", "sp_frac", "contrast_std"
          "all 10 (+ region features)": D.FEATURE_NAMES}
rows = []
for name, feats in groups.items():
    idx = [F[f] for f in feats]
    rf = RandomForestClassifier(300, random_state=0, n_jobs=-1).fit(X[tr][:, idx], y[tr])
    rows.append((name, len(feats), round(f1_score(y[te], rf.predict(X[te][:, idx]), average="m

# morphology ablation: recompute region features with a raw Otsu mask (no open/close, no large
def raw_mask(gr):
    _, m = cv2.threshold(gr, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU); return m
Xr = X.copy()
for i, p in enumerate(df.path):
    gr = D.to_grey(cv2.imread(os.path.join(DATA, p))); m = raw_mask(gr)
    a, c, f = D.region_feats(m); Xr[i, F["bbox_area"]], Xr[i, F["centroid_off"]], Xr[i, F["fg_
    Xr[i, F["bg_std"]] = D.bg_std(gr, m)
rf = RandomForestClassifier(300, random_state=0, n_jobs=-1).fit(Xr[tr], y[tr])
```

```
rows.append(("all 10, mask WITHOUT morphology/largest-CC", 10, round(f1_score(y[te], rf.predic
abl = pd.DataFrame(rows, columns=["feature set", "n_feat", "macro_F1"]); abl.to_csv(os.path.jo
print(abl)

# ---------------- block diagram
fig, ax = plt.subplots(figsize=(9, 2.6)); ax.axis("off")
boxes = [("Product image\n(256x256 BGR)", "#dfe8f5"), ("Pre-processing\ngrey, pad/resize", "#d
         ("DIP measures\nLaplacian var, MAD noise,\nhistogram spread, border", "#fbe5c8"),
         ("Segmentation\nOtsu + open/close +\nlargest CC, bbox", "#fbe5c8"),
         ("Classifier\nrules | RandomForest\n(vs CNN on pixels)", "#dcefd8"),
         ("Output\nPASS / FAIL + defect\ntype, metrics, failure cases", "#efe0f3")]
x = 0.01; w = 0.155
for i, (t, c) in enumerate(boxes):
    ax.add_patch(FancyBboxPatch((x, 0.2), w, 0.6, boxstyle="round,pad=0.01", fc=c, ec="k", lw=
    ax.text(x + w/2, 0.5, t, ha="center", va="center", fontsize=7.2)
    if i < len(boxes) - 1:
        ax.add_patch(FancyArrowPatch((x + w + 0.005, 0.5), (x + w + 0.035, 0.5), arrowstyle="-
    x += w + 0.04
ax.text(0.5, 0.05, "10-D feature vector  →  rule thresholds (baseline)  /  Random Forest (impr
        ha="center", fontsize=7, style="italic")
fig.savefig(os.path.join(RES, "block_diagram.png"), dpi=220, bbox_inches="tight"); plt.close(f
print("diagram ok")
```

results/metrics.json


```
{
  "results": [
    {
      "model": "Rule baseline (DIP features)",
      "accuracy": 0.913,
      "macro_f1": 0.9145,
      "per_class_f1": {
        "clean": 0.92,
        "blur": 0.953,
        "noise": 1.0,
        "low_contrast": 0.831,
        "bg_clutter": 0.966,
        "off_center": 0.817
      },
      "defect_recall": 0.9911,
      "false_reject_rate": 0.1111
    },
    {
      "model": "Random Forest (DIP features)",
      "accuracy": 0.9833,
      "macro_f1": 0.9833,
      "per_class_f1": {
        "clean": 0.967,
        "blur": 0.989,
        "noise": 1.0,
        "low_contrast": 0.96,
        "bg_clutter": 0.994,
        "off_center": 0.989
      },
      "defect_recall": 0.9933,
      "false_reject_rate": 0.0333
    },
    {
```

```
      "model": "Small CNN (raw pixels)",
      "accuracy": 0.9259,
      "macro_f1": 0.921,
      "per_class_f1": {
        "clean": 0.892,
        "blur": 0.972,
        "noise": 0.767,
        "low_contrast": 0.968,
        "bg_clutter": 0.932,
        "off_center": 0.994
      },
      "defect_recall": 0.96,
      "false_reject_rate": 0.0333
    }
  ],
  "meta": {
    "n_train": 1260,
    "n_test": 540,
    "feature_ms_per_img": 2.27,
    "rf_train_s": 0.52,
    "cnn_train_s": 65.5,
    "cnn_epochs": 20,
    "cnn_train_acc": 0.946,
    "rf_importance": {
      "sharpness": 0.3062,
      "border_white": 0.1929,
      "centroid_off": 0.0959,
      "hist_spread": 0.0815,
      "noise_mad": 0.0685,
      "sp_frac": 0.0635,
      "bbox_area": 0.0602,
      "bg_std": 0.0517,
      "fg_fill": 0.0412,
      "contrast_std": 0.0385
    }
  }
}```
```


results/ablation.csv

feature set,n_feat,macro_F1 sharpness only,1,0.7312 global stats (sharp+noise+contrast),5,0.9407 + background (border/bg_std),7,0.9523 all 10 (+ region features),10,0.9833 “all 10, mask WITHOUT morphology/largest-CC”,10,0.987
“‘